

BDDs

Binary Decision Diagrams. A way to compress functions (seen as a binary decision tree because the input parameters are only T/F) in regards to **memory**.

We first create a decision tree, kind of like in a boolean table for a function.

If you check the slides, we compare subtrees bottom up and link nodes to other subtrees (thereby creating a DAG), and wowzers, memory has become more efficient. This continues until you terminate.

Bounded model checking.

Bind checking to X amount of execution steps. In practice, essentially feels like verbose mode of normal NuSMV, but this way of running is nice because you can prove that something is safe for some bound (say, $k=20$ state transitions).

Fairness

A constraint that has to hold infinitely often during a valid trace (regarding temporal formulas).

A way to rule out “uninteresting” failure behaviour due to

Formal definition: Weak (Büchl) fairness $\rightarrow$ Always Eventually fairness in true implies that property is satisfied.

Basically, consider $S = \{a, b\}$, and $d = \{(a,a), (a,b), (b,b)\}$ You can loop on a infinitely, but because we are fair, we will go to b.

Strong (Streett) fairness $\rightarrow$ Not covered by course, but kind of same as above, it's just that we also consider loops that span multiple states.

Note of course, fairness in this case only considers that we eventually break some loop, but it's not really equivalent with what we would consider fair in say, operating system scheduling or a traffic light (which would maybe utilize timers instead).

BEWARE DEADLOCKS. BEWARE SILLY FAIRNESS PROPERTIES. In the example, the top one is bad because then, we require that ALL states